

Homework deliverable — Airspace Intelligence System

Table of contents

- 1. Writing component (your own words; not AI-generated)
- 2. Git repository links (paste your GitHub URLs)
- 3. Screenshots (local run acceptable)
 - 3.1 Multi-agent workflow + tool usage (Agent 1 — live data / tools_used)
 - 3.2 RAG retrieval and response (Agent 2 — reference)
 - 3.3 Synthesis (Agent 3)
 - 3.4 Final report (Agent 4)
- 4. Documentation
 - 4.1 System architecture — agent roles and workflow
 - 4.2 RAG data source and search
 - 4.3 Tool functions (name, purpose, parameters, returns)
 - 4.4 Technical details
 - 4.5 Usage instructions
- License and disclaimers

1. Overview

This Airspace Intelligence System takes inputs from the user pertaining to an upcoming flight (carrier, flight #, departing/arriving airports) and returns a concise summary of the risk of flight delay/cancellation based on factors like FAA ground stops, temporary flight restrictions, and nationwide aviation weather patterns. The app fills a gap in real-time communication for travelers affected by flight issues. I have personally experienced significant distress while flying over the past year due to poor communication from airlines about the causes of delays/cancellations and their likelihood of resolution. The primary impetus to develop this app came a couple weeks ago while working on one of this class's assignments in an airport hotel room at DFW due to a last minute flight cancellation. Significant storm systems around the country coupled with compounding delays and ground stops led to my second flight of the day getting delayed repeatedly and ultimately cancelled. A tool like this would have allowed me to understand these risks at the start of the day and take mitigation actions like packing toiletries in my backpack and not checking my carry-on; instead, I found myself stuck in an airline help desk queue trying to understand what was going on. Tools like FlightAware's MiseryMap are helpful to visualize airspace system disruptions, but they offer no information about root causes of disruptions, nor a direct application to my own flight's risk.

The overall app architecture uses an agent orchestrator to cue 4 different agents, including tool calls from an MCP server. The first agent calls API tools to compile available national airspace and weather data. The second agent runs in parallel and conducts BM25 keyword search-based RAG over FAA airport information repositories that have been pre-chunked. The third agent takes information from both the prior agents and builds a national airspace network synthesis; the fourth agent uses all prior inputs to build a personalized flight report. Results of all 4 agents are available for the user to view to trace the insights back to their data sources.

I had the most trouble with the RAG agent to ensure repeatable results. I had to adjust the hit count to ensure the RAG search would run through all the data available and in a repeatable order. Chunks are now managed by a standardized chunk_id, BM25 scores are maxed across queries, and results are sorted by scoring. API tool queries were straightforward for the most part. I set up the tools in a separate MCP server to get practice working with it; for this purpose, in-process tools could have been used. Indeed, not setting an MCP server URL will prompt the app to default to in-process tool calls. I also pursued deployment to Posit to enable beta testing by friends and family.

Potential Improvements for a Future Revision

- Several beta testers reported the app takes way too long to run (on the order of 2-3 minutes) and can sometimes result in server timeout before getting a response.
- One solution to the above could be to try a more lightweight model to be more mobile/ADHD-friendly
- Better types of recommendations that aren't just common sense. Information like "beware if you have a connection through DFW" would be valuable even if the flight queried did not mention that airport.
- Scrap the 4th agent if it is possible to bake in the personal aspect into the Agent 3 airspace synthesis.
- More attractive UI and some explanation text at the top of the display
- Add back TSA wait times once that API works again

- Find a free API to pull flight information just based on flight # and date to minimize human inputs needed
- Find a free API to get "where is my plane?" information
- FAA RAG data is set up as a static PDF repository - could look into ways to automatically pipeline in new document revisions when they are released
- See if there is public information about historical on time data for a given flight number that could factor into Agent 4's risk score
- Consider a low cost paid service instead of Ollama to get faster agent runtimes

2. Git repository links

Required artifact	File in this repo (relative path)	Link to GitHub Online
Multi-agent orchestration	App V3 For Deployment/app/agents/orchestrator.py	https://github.com/JdogLloyd1/Agentic-Flight-Report/blob/main/App%20V3%20For%20Deployment/app/agents/orchestrator.py
RAG implementation	App V3 For Deployment/app/rag/search.py	https://github.com/JdogLloyd1/Agentic-Flight-Report/blob/main/App%20V3%20For%20Deployment/app/rag/search.py
Function calling / tool definitions	App V3 For Deployment/mcp_server/tools/registry.py	https://github.com/JdogLloyd1/Agentic-Flight-Report/blob/main/App%20V3%20For%20Deployment/mcp_server/tools/registry.py
Main system / UI	App V3 For Deployment/app/shiny_app.py	https://github.com/JdogLloyd1/Agentic-Flight-Report/blob/main/App%20V3%20For%20Deployment/app/shiny_app.py

Optional extras (deployment / bridge):

Topic	File	Link
MCP HTTP bridge (Connect)	App V3 For Deployment/mcp_server/http_bridge.py	https://github.com/JdogLloyd1/Agentic-Flight-Report/blob/main/App%20V3%20For%20Deployment/mcp_server/http_bridge.py
Deploy scripts	App V3 For Deployment/scripts/deploy_shiny.py , deploy_mcp_http_bridge.py	https://github.com/JdogLloyd1/Agentic-Flight-Report/blob/main/App%20V3%20For%20Deployment/scripts/deploy_shiny.py , https://github.com/JdogLloyd1/Agentic-Flight-Report/blob/main/App%20V3%20For%20Deployment/scripts/deploy_mcp_http_bridge.py

3. Screenshots

Captured from the local Shiny app (`python app/run_me.py` in **App V3 Local Run** , default <http://127.0.0.1:8000>). Paths are **relative to this file** at repo root.

3.1 Multi-agent workflow + tool usage (Agent 1 — live data / tools_used)

Structured JSON from tool calls (NAS, weather, etc.).

Airspace Intelligence Agent

Multi-agent workflow: live NAS/weather tools (Agent 1), FAA reference search (Agent 2), synthesis (Agent 3), and personalized advice (Agent 4).
Ollama Cloud requires OLLAMA_API_KEY in .env.

Carrier (IATA)	Flight number	Flight date	Origin (ICAO/IATA)	Destination
AA	2308	2026-04-01	RDU	LAX

Run analysis

Done.
Step 4 of 4 — Complete. Results are in the tabs below.

Final report (Agent 4) Synthesis (Agent 3) Reference (Agent 2) Live data (Agent 1) Errors / trace

```
{
  "airports": {
    "origin_icao": "KRDU",
    "destination_icao": "KLAX"
  },
  "nas_status_faa": {
    "status": "ok",
    "summary": "NASSTATUS update Thu Apr 2 00:27:49 2026 GMT. Ground Stop Programs: DCA, LGA, TEB, EWR, BWI (thunderstorms, end times 9:00-9:15pm EDT). Ground Delay Programs: JFK avg 61min max 2:07, DCA avg 45min max 3:37, EWR avg 46min max 1:42, LGA avg 1:48 max 13:39 (thunderstorms). Arrival/Departure Delays: BWI 2:01-2:15 increasing, IAD 1:46-2:00 increasing, JFK 31-45min increasing, MCO 16-30min, MIA 46-60min (ROCKET LAUNCH ARTEMIS II), ORD 16-30min (Multi-taxi), PHL 16-30min, TEB 16-30min. Airport Closures: SNA (DLY 0630-1315 thru Apr 4), STX (GA restrictions thru Apr 30), SAN (GA restrictions thru Oct 1), LAS (GA restrictions thru Apr 29). No GDP/GS directly affecting KRDU or KLAX.",
    "tools_used": [
      "fetch_nasstatus_airport_status"
    ]
  },
  "operational_weather_aws_nws": {
    "status": "ok",
    "origin_metar_taf": "KRDU METAR 012351Z 18006KT 10SM FEW060 BKN250 27/14 A3011 VFR. TAF 012336Z 0200/0224 20009KT P6SM FEW060 SCT250 TEMPO 0213/0214 BKN060 010015 FM021500 20010KT 30000018KT BKN060 SCT250 0216-0217 12-14Z with low clouds 000-1500ft."
  }
}
```

http://127.0.0.1:8000/#tab-616771-3

3.2 RAG retrieval and response (Agent 2 — reference)

FAA / facility excerpts from BM25 search.

Airspace Intelligence Agent

Multi-agent workflow: live NAS/weather tools (Agent 1), FAA reference search (Agent 2), synthesis (Agent 3), and personalized advice (Agent 4).
Ollama Cloud requires OLLAMA_API_KEY in .env.

Carrier (IATA)	Flight number	Flight date	Origin (ICAO/IATA)	Destination
AA	2308	2026-04-01	RDU	LAX

Run analysis

Done.
Step 4 of 4 — Complete. Results are in the tabs below.

- Final report (Agent 4)
- Synthesis (Agent 3)
- Reference (Agent 2)
- Live data (Agent 1)
- Errors / trace

- Ground Delay Programs (GDP) and Ground Stops (GS):** GDPs are traffic management processes administered by the ATCSCC to hold aircraft on the ground to limit airborne holding, while GSs require aircraft meeting specific criteria (e.g., destination-specific) to remain on the ground, often with little warning. Controllers must not release aircraft during an applicable GS without originator approval (faa_7110_65.pdf).
- Hazardous Weather Advisories:** Controllers are required to advise pilots of hazardous weather (SIGMET, Convective SIGMET, AIRMET, CWA) impacting operations within 150 NM of their sector. Specific values (ceiling/visibility) must be obtained from official sources or FAA-approved automation before transmission (faa_7110_65.pdf).
- LAX Noise Abatement and Runway Use:** During daytime/evening (0700L–2200L), LAX prefers outer runways (24R/25L) for arrivals and inner runways (24L/25R) for departures. Between 2400L–0630L, aircraft should approach and depart over the ocean unless unsafe due to weather or equipment (airport_facilities/sw_LAX_notices_19MAR2026.pdf , airport_facilities/SW_rear_19MAR2026.pdf).
- Traffic Flow Management (TBFM/TBM):** Arrival into high-density terminals like LAX may involve Time-Based Flow Management (TBFM) metering. Pilots should expect compliance with metering times within +/- 1 minute and potential miles-in-trail (MIT) restrictions to manage demand/capacity imbalances (faa_7110_65.pdf).
- Data Limitation (RDU):** While the AFD index confirms RDU facility data effective for the flight date (airport_facilities/afd_19MAR2026.xml), the retrieved excerpts lack specific RDU departure procedure notes or local traffic management initiatives compared to the detailed LAX notices.

http://127.0.0.1:8000/#tab-616771-2

3.3 Synthesis (Agent 3)

Network-level merge of live data and reference context.

Airspace Intelligence Agent

Multi-agent workflow: live NAS/weather tools (Agent 1), FAA reference search (Agent 2), synthesis (Agent 3), and personalized advice (Agent 4).
Ollama Cloud requires OLLAMA_API_KEY in .env.

Carrier (IATA)	Flight number	Flight date	Origin (ICAO/IATA)	Destination
AA	2308	2026-04-01	RDU	LAX

Run analysis

Done.
Step 4 of 4 — Complete. Results are in the tabs below.

- Final report (Agent 4)
- Synthesis (Agent 3)
- Reference (Agent 2)
- Live data (Agent 1)
- Errors / trace

1) NAS Status (FAA)

Current Operational Status (Live)

- Overall NAS Status: OK (as of Thu 2 00:27:49 2026 GMT).
- Route Specifics (KRDU → KLAX): No Ground Delay Programs (GDP) or Ground Stops (GS) directly affecting origin (KRDU) or destination (KLAX).
- Active Traffic Management Programs:
 - Ground Stops (GS): DCA, LGA, TEB, EWR, BWI (Reason: Thunderstorms; End Times: 9:00-9:15pm EDT).
 - Ground Delay Programs (GDP): JFK (Avg 61min), DCA (Avg 45min), EWR (Avg 46min), LGA (Avg 1:48).
 - Arrival/Departure Delays: BWI (2:01-2:15 increasing), IAD (1:46-2:00 increasing), JFK (31-45min increasing), MIA (46-60min), ORD (16-30min), PHL (16-30min).

Regulatory Context (Ref)

- GDP/GS Definitions: GDPs are administered by ATCSCC to hold aircraft on the ground to limit airborne holding. GSs require aircraft meeting specific criteria to remain on the ground; controllers must not release aircraft during an applicable GS without originator approval (faa_7110_65.pdf).
- Implication: While KRDU/KLAX are clear, connecting traffic through Northeast hubs (JFK/EWR/LGA/DCA) will face significant regulatory holds.

2) Current Operational Weather (AWC / NWS)

http://127.0.0.1:8000/#tab-616771-1

3.4 Final report (Agent 4)

Personalized flight report (workflow complete).

Airspace Intelligence Agent

Multi-agent workflow: live NAS/weather tools (Agent 1), FAA reference search (Agent 2), synthesis (Agent 3), and personalized advice (Agent 4). Ollama Cloud requires OLLAMA_API_KEY in .env.

Carrier (IATA)	Flight number	Flight date	Origin (ICAO/IATA)	Destination
AA	2308	2026-04-01	RDU	LAX

Run analysis

Done.

Step 4 of 4 — Complete. Results are in the tabs below.

Final report (Agent 4)

Synthesis (Agent 3)

Reference (Agent 2)

Live data (Agent 1)

Errors / trace

Risk Level: Moderate **Rationale:** While both RDU and LAX airports are currently operating without ground stops, significant severe thunderstorm activity along the central enroute corridor may cause turbulence or minor routing delays.

Key Risk Drivers

- **Enroute Weather:** There are 12 active Convective SIGMETs covering the central U.S. (TX, OK, CO, NM, etc.), indicating severe thunderstorms, hail, and turbulence that may require flight path deviations.
- **Turbulence:** Severe turbulence has already been reported near the origin (RDU), and enroute weather cells pose a continued risk of bumpiness during cruise.
- **Arrival Flow:** Although LAX has no formal delays, high volume and weather avoidance may trigger traffic metering, potentially adding time to the approach.
- **Security Checkpoints:** Automated TSA wait time data is unavailable for RDU, creating uncertainty regarding security line length.

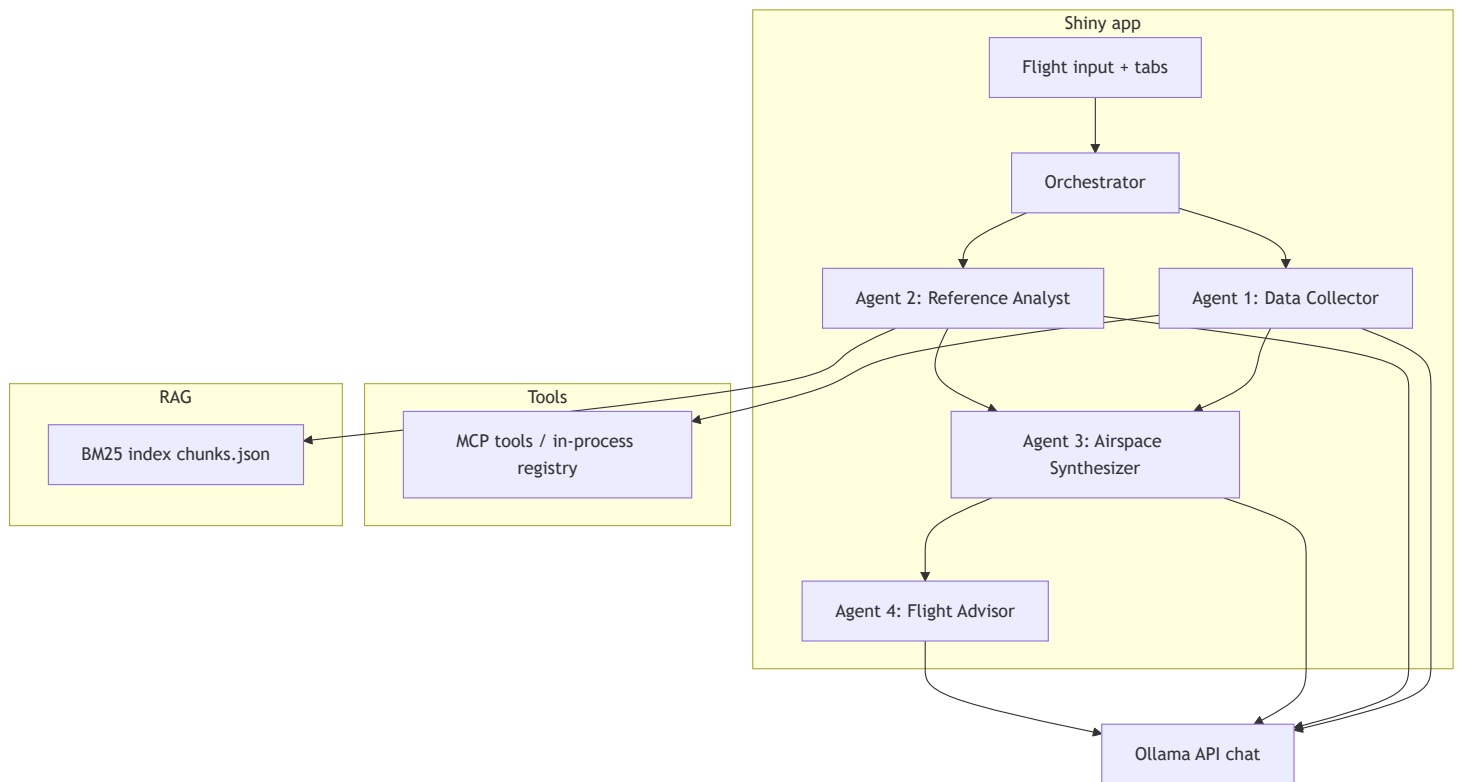
Practical Recommendations

- **Monitor Flight Status:** Check your flight status frequently on the day of travel, as enroute weather can lead to last-minute gate changes or slight schedule adjustments.
- **Arrive Early:** Plan to arrive at RDU at least 2 hours before departure; verify current TSA wait times manually at [tsa.gov](https://www.tsa.gov) since automated data is unavailable.
- **Prepare for Turbulence:** Keep your seatbelt fastened whenever seated and secure loose items in case the crew needs to deviate around storm cells.

4. Documentation

4.1 System architecture — agent roles and workflow

Four agents; Agents **1** and **2** run **in parallel**, then **3**, then **4**. The Shiny UI drives `app/shiny_app.py` and the orchestrator in `app/agents/orchestrator.py`.



Agent	Role
1 — Data Collector	Calls registered tools in a loop via Ollama chat; live NAS, weather, TFRs, etc.
2 — Reference Analyst	BM25 queries from flight context; ranked FAA / facility excerpts.
3 — Airspace Synthesizer	Merges Agent 1 + 2 into a network-level narrative.
4 — Flight Advisor	Risk, drivers, recommendations, disclaimers for the specific flight.

Split deployment (Posit Connect): With `MCP_BASE_URL` set, Agent 1 calls `POST {MCP_BASE_URL}/tools/call` on the HTTP bridge; with it empty, tools run **in-process** in the Shiny worker (same as local default).

4.2 RAG data source and search

All raw files live under `app/rag/data/` (or whatever you set in `RAG_DATA_DIR`). Ingestion is implemented in `app/rag/ingest.py`.

Data Sources

Kind	Location	What gets indexed
FAA order / procedures PDFs	*.pdf in the data root	Full text extracted with PyMuPDF (<code>fitz</code>), then split into chunks (preferring lines that look like section headers; otherwise sliding windows). Typical content is FAA order-style reference material (e.g. air traffic / procedures text you place in the bundle).
Airport Facility Directory (AFD) index XML	airport_facilities/*.xml	d-TPP-style airport index: root <code><airports></code> , then <code><location></code> / <code><airport></code> rows with state , city , LID (<code>aptid</code>), NAVAID names, and chart PDF filename(s) plus effective-date metadata. Each row becomes one BM25 chunk with human-readable fields and pointers to PDFs; the XML does not contain the full chart text.
AFD / chart PDFs	airport_facilities/*.pdf (and any root-level PDFs above)	Same PDF pipeline as order PDFs: all pages concatenated, then chunked. These align with filenames referenced from the XML index where you ship both.

Index artifacts (written under `app/rag/data/index/`):

- `chunks.json` — unified BM25 corpus (XML rows + all ingested PDF text).

- `airport_index.json` — maps **uppercase LID / NAVAID** strings to `chunk_id` s for hybrid lookup alongside BM25 (see `app/rag/search.py`).

Topic	Detail
Ingestion	<code>python -m app.rag.ingest</code> runs the scan above and writes <code>index/chunks.json</code> and <code>index/airport_index.json</code> . Run after adding or changing PDFs/XML; run before bundling for Connect so the bundle ships a fresh index.
Search	<code>search_reference(query, top_k=...)</code> — BM25 (<code>rank_bm25</code>), not dense embeddings, with light tokenization/stop-word handling. Returns ranked dicts with fields such as <code>source</code> , <code>section</code> , <code>content</code> , <code>score</code> (and <code>chunk_id</code> where present).

4.3 Tool functions

Tool	Purpose	Main parameters	Returns (summary)
<code>fetch_nasstatus_airport_status</code>	FAA NAS airport status (GDPs, ground stops)	<code>url?</code> , <code>include_parsed_json?</code>	JSON: NAS delay data
<code>get_metar</code>	METAR for an ICAO station	<code>station_id</code> , <code>hours_back?</code>	JSON with observation text
<code>get_taf</code>	TAF for an ICAO station	<code>station_id</code>	JSON with forecast text
<code>get_sigmet</code> s	SIGMETs	<code>hazard_type?</code>	JSON array of advisories
<code>get_gairmet</code> s	G-AIRMETs	<code>hazard_type?</code>	JSON array
<code>get_pirep</code> s	PIREPs	<code>station_id?</code> , <code>distance_nm?</code>	JSON array
<code>get_weather_alerts</code>	NWS active alerts	<code>area?</code> , <code>severity?</code>	JSON alerts
<code>get_active_tfrs</code>	Active TFRs	<code>max_features?</code> , <code>include_geometry?</code>	JSON features
<code>get_tsa_wait_times</code>	TSA waits (proxy supported)	<code>airport_code</code>	JSON / error JSON

Agent 1's default allowlist is configured in `mcp_server/tools/registry.py` (`DEFAULT_AGENT_TOOL_SCHEMAS` vs `ALL_TOOL_SCHEMAS`).

4.4 Technical details

Packages / runtime

- Local:** Python 3.10+; `pip install -r requirements.txt` from App V3 Local Run .
- Connect bundle:** Python **3.11** recommended; see App V3 For Deployment/runtime.txt and requirements.txt (includes `rsconnect-python`).

APIs and hosts

- LLM:** Ollama Cloud or local Ollama — `OLLAMA_API_KEY` , `OLLAMA_HOST` , `OLLAMA_MODEL` , `OLLAMA_TEMPERATURE` .
- Live data:** FAA / NOAA / NWS / TFR feeds; optional OpenSky (`OPENSKY_*`), TSA proxy (`TSA_WAIT_TIMES_PROXY_URL`).

Live deployment (Cornell Systems / Posit Connect) — from [root](#) README.md :

- Shiny app:** <https://connect.systems-apps.com/connect/#/apps/e115a8d7-42c7-439d-9466-d1c8e698a9e8>
- MCP HTTP bridge (Swagger):** <https://connect.systems-apps.com/content/07bcfaa3-9ce7-41bf-adfe-845083766c0e>

Repo layout (high level)

Path	Role
App V3 For Deployment/app/shiny_app.py	Shiny UI
App V3 For Deployment/app/agents/	Orchestrator + agents
App V3 For Deployment/app/rag/	Ingest, search, data

Path	Role
App V3 For Deployment/mcp_server/	Tools, HTTP bridge, FastMCP
App V3 For Deployment/scripts/	deploy_shiny.py , deploy_mcp_http_bridge.py

Deeper product notes: [PLAN.md](#) .

4.5 Usage instructions

Run locally (development)

- 1. cd into App V3 Local Run .
- 2. pip install -r requirements.txt
- 3. Copy .env.example → .env and set Ollama (and optional MCP_BASE_URL , RAG_DATA_DIR , etc.).
- 4. Optional: python -m app.rag.ingest after changing RAG documents.
- 5. python app/run_me.py → open <http://127.0.0.1:8000>

Deploy to Posit Connect

- 1. Use App V3 For Deployment : pip install -r requirements.txt , configure .env with CONNECT_SERVER and POSIT_CONNECT_PUBLISHER or CONNECT_API_KEY .
- 2. Typical order: deploy **MCP HTTP bridge** (python scripts/deploy_mcp_http_bridge.py), set MCP_BASE_URL on the Shiny content, deploy **Shiny** (python scripts/deploy_shiny.py).
- 3. Set Ollama vars on each Connect content item (**Vars** tab). Ensure RAG index is built before bundle (python -m app.rag.ingest).

Local smoke test (HTTP bridge)

```
python -m uvicorn mcp_server.http_bridge:app --host 127.0.0.1 --port 8766
```

Set MCP_BASE_URL=http://127.0.0.1:8766 and run Shiny.

License and disclaimers

Outputs are **AI-generated** and **not** official FAA or airline information. Verify operational decisions with dispatch, ATC, and your carrier.